

PYTHON Problems and Solutions

async scraper solution

```
# async_scraper_solution.py
import asyncio
import aiohttp

async def fetch(session, url):
    async with session.get(url) as r:
        return await r.text()

async def main(urls):
    async with aiohttp.ClientSession() as s:
        tasks = [fetch(s,u) for u in urls]
        return await asyncio.gather(*tasks)

if __name__=='__main__':
    import sys
    urls = ['https://example.com'] if len(sys.argv)==1 else sys.argv[1:]
    res = asyncio.run(main(urls)); print([len(r) for r in res])
```

async scraper starter

```
# async_scraper_starter.py
import asyncio
# use aiohttp to fetch multiple urls concurrently
```

fib gen solution

```
# fib_gen_solution.py
def fib():
    a,b = 0,1
    while True:
        yield a
        a,b = b, a+b

if __name__=='__main__':
    f = fib()
    for _ in range(10): print(next(f))
```

fib gen starter

```
# fib_gen_starter.py
def fib():
    # TODO: generator
    pass

if __name__=='__main__':
    f = fib()
    for _ in range(10): print(next(f))
```

fifo queue solution

```
# fifo_queue_solution.py
class FIFOQueue:
    def __init__(self):
```

```

        self.queue = []
    def enqueue(self, item):
        self.queue.append(item)
    def dequeue(self):
        if not self.queue: raise IndexError('dequeue from empty')
        return self.queue.pop(0)
    def peek(self):
        return self.queue[0] if self.queue else None

if __name__ == '__main__':
    q = FIFOQueue(); q.enqueue(1); q.enqueue(2); q.enqueue(3)
    print(q.dequeue()); q.enqueue(4); print(q.peek())

```

fifo queue starter

```

# fifo_queue_starter.py
class FIFOQueue:
    def __init__(self):
        self.queue = []
    def enqueue(self, item):
        pass
    def dequeue(self):
        pass
    def peek(self):
        pass

```

freq counter solution

```

# freq_counter_solution.py
import re
from collections import Counter
def word_freq(text):
    words = re.findall(r"\w+", text.lower())
    return Counter(words)

if __name__ == '__main__':
    print(word_freq('This is a test. This test is simple.'))

```

freq counter starter

```

# freq_counter_starter.py
def word_freq(text):
    # TODO
    pass

```

lru cache solution

```

# lru_cache_solution.py
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity; self.od = OrderedDict()
    def get(self, key):
        if key not in self.od: return -1
        self.od.move_to_end(key); return self.od[key]
    def put(self, key, value):
        if key in self.od: self.od.move_to_end(key)
        self.od[key]=value
        if len(self.od)>self.cap: self.od.popitem(last=False)

if __name__ == '__main__':
    c=LRUCache(2); c.put(1,1); c.put(2,2); print(c.get(1)); c.put(3,3); print(c.get(2))

```

lru cache starter

```
# lru_cache_starter.py
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity): pass
    def get(self, key): pass
    def put(self, key, value): pass
```

matrix rot solution

```
# matrix_rot_solution.py
def rotate90(m):
    n = len(m)
    for i in range(n//2):
        for j in range(i, n-1-i):
            m[i][j], m[j][n-1-i], m[n-1-i][n-1-j], m[n-1-j][i] = m[n-1-j][i], m[i][j], m[j][n-1-i], m[n-1-i][n-1-j]
if __name__ == '__main__':
    m=[[1,2,3],[4,5,6],[7,8,9]]; rotate90(m); print(m)
```

matrix rot starter

```
# matrix_rot_starter.py
def rotate90(m):
    # TODO in-place rotate NxN matrix
    pass
```

merge sorted solution

```
# merge_sorted_solution.py
def merge(a,b):
    i=j=0; out=[]
    while i<len(a) and j<len(b):
        if a[i]<=b[j]: out.append(a[i]); i+=1
        else: out.append(b[j]); j+=1
    out.extend(a[i:]); out.extend(b[j:]); return out
if __name__ == '__main__': print(merge([1,3,5],[2,4,6]))
```

merge sorted starter

```
# merge_sorted_starter.py
def merge(a,b):
    # TODO
    pass
```

palindrome solution

```
# palindrome_solution.py
import re
def is_palindrome(s):
    s2 = re.sub(r'^A-Za-z0-9','', s).lower()
    return s2 == s2[::-1]
if __name__ == '__main__': print(is_palindrome('racecar'))
```

palindrome starter

```
# palindrome_starter.py
def is_palindrome(s):
    # TODO
    pass

if __name__=='__main__': print(is_palindrome('racecar'))
```

prod cons solution

```
# prod_cons_solution.py
import threading, time, queue

def producer(q):
    for i in range(5):
        q.put(i); print('prod',i); time.sleep(0.1)
def consumer(q):
    while True:
        item = q.get(); print('cons',item); q.task_done()

if __name__=='__main__':
    q = queue.Queue()
    t1 = threading.Thread(target=producer,args=(q,)); t2 = threading.Thread(target=consumer,args=(q,))
    t1.start(); t2.daemon=True; t2.start(); t1.join(); q.join()
```

prod cons starter

```
# prod_cons_starter.py
import threading, time
# Implement producer-consumer with queue
```

quicksort solution

```
# quicksort_solution.py
def quicksort(a):
    if len(a)<=1: return a
    pivot=a[len(a)//2]; left=[x for x in a if x<pivot]; mid=[x for x in a if x==pivot]; right=[x for x in a if x>pivot]
    return quicksort(left)+mid+quicksort(right)
if __name__=='__main__': print(quicksort([3,6,8,10,1,2,1]))
```

quicksort starter

```
# quicksort_starter.py
def quicksort(a): pass
```